

LU2IN006 - Structures de données

Projet: Réalisation d'un simulateur de CPU

Travaille par

Ermal ZEQO 21315866

Fitia SENDRISON 21318175

Professeurs

Jean-Baptiste Deloges

Sebastien Lallé

Année académique 2024-2025

Abstrait

Le processeur, ou unité centrale de traitement (CPU - Central Processing Unit), est chargé d'exécuter les instructions d'un programme. Il assure la coordination de l'accès à la mémoire, réalise les calculs nécessaires et contrôle le déroulement de l'exécution des programmes. Son fonctionnement repose sur un cycle d'instruction durant lequel il va chercher une instruction en mémoire, la décode, puis l'exécute en manipulant les registres et la mémoire.

Le processeur se compose de plusieurs éléments essentiels :

- **Les registres** : ce sont de petites zones de mémoire interne au processeur, utilisées pour stocker temporairement des données utiles aux calculs et à l'exécution des instructions. On y trouve notamment les registres généraux (AX, BX, CX, DX), les registres d'indexation, et les registres de contrôle (IP, SP, FLAGS).
- **L'unité de contrôle** : elle dirige l'exécution des instructions en prenant en charge leur récupération, leur décodage et leur exécution.
- **L'unité arithmétique et logique (ALU)** : elle effectue les opérations mathématiques (addition, soustraction, etc.) et logiques (comparaisons, etc.).
- **L'accès à la mémoire** : le processeur communique avec la mémoire vive (RAM) pour charger les instructions et stocker temporairement des données. Il utilise plusieurs modes d'adressage pour optimiser l'accès à ces données.

Dans ce projet, on veut implémenter une version simplifiée de ce fonctionnement. L'accent sera mis sur un nombre limité de registres et de modes d'adressage, tout en conservant les principes fondamentaux liés au traitement des instructions et à la gestion de la mémoire.

Table des matières

1-Implémentation d’une table de hachage générique

- 1.1 unsigned long simple hash(const char *str)
- 1.2 Création d'une hashmap
- 1.3 Malloc vs Calloc
- 1.4 Insertion d’un élément dans hashmap
- 1.5 Récupération d’un eléments
- 1.6 Suppression d’une clé d'une table de hachage
- 1.7 Nettoyage de la mémoire utilisée
- 1.8 Main d’exercice 1

2-Gestion dynamique de la mémoire

- 2.1 Initialisation du gestionnaire de mémoire
- 2.2 Recherche d’un segment libre contenant une certaine plage mémoire
- 2.3 Allocation d’un segment nommé à une position donnée
- 2.4 Libération d’un segment nommé et fusion avec les segments libres
- 2.5 Libération complète de la mémoire du gestionnaire

3-Conception d’un parser pour un langage pseudo-assembleur

- 3.1 Fonction pour parser une instruction de la section .DATA
- 3.2 free_parser_result
- 3.3 parse
- 3.4 free_parser_result

4-Allocation d’un segment de données

- 4.1 Fonction permettant d'initialiser le processeur
- 4.2 Libération de toutes les ressources allouées
- 4.3 Stocker data

4.4 Récupération des données stockées

4.5 Allocation dynamiquement le segment de données

4.6 L'afficher le contenu

5-Expressions régulières et résolution d’adressage

5.1 Initialisation de la CPU2

5.2 Destruction de la CPU2

5.3 Adressage immédiat

5.4 Adressage par registre

5.5 Adressage direct en mémoire

5.6 Adressage indirect par registre

5.7 Gestion de l'instruction MOV

5.8 Résolution d'adressage

6-Allocation d’un segment de codes et exécution

6.1 Trim

6.2 search_and_replace

6.3 resolve_constants

6.4 cpu_inite

6.5 allocate_code_segment

6.4 handle_instruction

6.6 execute_instruction

6.7 fetch_next_instruction

6.8 run_program

7-Implémentation d’un segment de pile (Stack Segment)

7.1 Initialisation de la pile (Stack)

7.2 Empiler une valeur

7.3 Dépiler une valeur

8-Gestion de l’allocation dynamique (Extra Segment)

- 8.1 segment_override_addressing
- 8.2 resolve_addressing_2
- 8.3 find_free_address_strategy
- 8.4 ajout_ES_dans_cpu_init
- 8.5 alloc_es_segment
- 8.6 free_es_segment
- 8.7 handle_instruction_extended

1. Implémentation d'une table de hachage générique

Dans cet exercice, nous allons implémenter une table de hachage générique en C. L'objectif est de créer une structure permettant d'associer des clés (chaînes de caractères) à des valeurs de type générique void*. Cette flexibilité nous permettra de concevoir un dictionnaire associant des clés à n'importe quel type de structure. La table de hachage utilisera une taille fixe de TABLE_SIZE = 128 et appliquera un mécanisme de sondage/probing linéaire en cas de collision. Ainsi on utilisera les structures suivantes :

typedef struct hashentry { char * key ; void * value ; } HashEntry ;	typedef struct hashmap { int size ; HashEntry * table ; } HashMap ;

Remarque : Lorsqu'on supprime un élément dans une table de hachage utilisant l'adressage ouvert, il est courant d'utiliser une **TOMBSTONE** afin de ne pas rompre les chaînes de sondage (probing). Une TOMBSTONE est une marque spéciale, différente d'une case vide (NULL), qui remplace un élément supprimé.

Contrairement à NULL, elle permet de continuer correctement les recherches, en garantissant que les éléments insérés plus loin dans la chaîne restent accessibles. Bien entendu, si une nouvelle insertion doit se faire à l'emplacement d'une TOMBSTONE, celle-ci est remplacée par le nouvel élément.

Dans notre cas, on pourra par exemple utiliser une constante spéciale pour représenter une TOMBSTONE, définie comme suit :

```
# define TOMBSTONE (( void *) -1)
```

Q 1.1 unsigned long simple hash(const char *str)

```c
unsigned long simple_hash(const char *str) {
    unsigned long hash = 5381;
    int c;
    for (; *str; str++) {
        c = *str;
        hash = ((hash << 5) + hash) + c;
    }
    return hash % TABLE_SIZE;
}
```

Ce code permettant de convertir une chaîne de caractères en un indice dans la table de hachage.
On utilise 5381 qui été choisi pour donner de bons résultats dans la fonction `hash = hash * 33 + c`, en particulier pour des chaînes ASCII.

hash << 5

➜ décalage à gauche de 5 bits, équivalent à hash * 32.

((hash << 5) + hash) + c

➜ donc hash = hash * 33 + c

((hash << 5) + hash)

➜ donc hash * 32 + hash = hash * 33.

hash = ((hash << 5) + hash) + c; est une façon rapide d'écrire hash = hash * 33 + c;.

Q1.2 Création d'une hashmap

```
HashMap* hashmap_create() {
    HashMap* map = (HashMap*) malloc(sizeof(HashMap));
    if (!map) return NULL;
    map->size = TABLE_SIZE;
    map->table = (HashEntry*) calloc(TABLE_SIZE, sizeof(HashEntry));
    if (!map->table) {
        free(map);
        return NULL;
    }
    return map;
}
```

Cette fonction crée une nouvelle table de hachage (HashMap) et l'initialise correctement en mémoire.
Elle retourne un pointeur vers cette nouvelle table ou NULL si ça a échoué. On fixe la taille de la table de hachage. On réserve un tableau de TABLE_SIZE entrées de type HashEntry, initialisées à zéro grâce à calloc. Si l'allocation du tableau échoue, on libère ce qu'on a déjà alloué (map) pour éviter une fuite mémoire, et on retourne NULL.

1.3 Malloc vs Calloc

malloc (memory allocate)

void* malloc(size_t size);

- Elle réserve de la mémoire brute, de la taille indiquée (en octets).
- Elle ne l'initialise pas. Le contenu est aléatoire (peut contenir n'importe quoi).
- Exemple : int* tab = (int*) malloc(5 * sizeof(int));

Le tableau tab à 5 cases, **non initialisées** (valeurs indéfinies).

calloc (contiguous allocate)

void* calloc(size_t n, size_t size);

- Elle réserve de la mémoire pour n éléments de taille size chacun.
- Elle initialise toute la mémoire à 0.
- Exemple : int* tab = (int*) calloc(5, sizeof(int));

Le tableau tab a 5 entiers, **tous initialisés à zéro**.

1.4 Insertion d'un élément dans hashmap

```c
char* strdup(const char* str) {
    size_t len = strlen(str) + 1;
    char* copy = (char*)malloc(len);
    if (copy) {
        memcpy(copy, str, len);
    }
    return copy;
}
int hashmap_insert(HashMap *map, const char *key, void *value) {
    unsigned long index = simple_hash(key);
    for (int i = 0; i < TABLE_SIZE; i++) {
        unsigned long new_index = (index + i) % TABLE_SIZE;
        if (map->table[new_index].key == NULL || map->table[new_index].key == TOMBSTONE) {
            map->table[new_index].key = strdup(key);
            map->table[new_index].value = value;
            printf("Insertion de la clé : %s à l'index %lu\n", key, new_index);
            return 1;
        }
    }
    return 0;
}
```

La fonction strdup fait une copie dynamique d'une chaîne de caractères. Elle alloue de la mémoire pour une nouvelle chaîne. Copie le contenu d'une str dans cette nouvelle mémoire. Toujours utile pour dupliquer une clé avant de la stocker dans la table.

La fonction `hashmap_insert` ajouter une nouvelle paire clé/valeur dans la table. Elle commence par calculer où placer la clé avec une fonction de hachage. Ensuite, elle cherche une case libre à partir de cet indice. Si la case est vide ou si elle contenait une ancienne valeur supprimée (on utilise alors une sorte de marque spéciale qu'on appelle TOMBSTONE), alors elle place la nouvelle clé à cet endroit et y met la valeur associée. La clé est copiée pour éviter les soucis avec la mémoire.

Si toutes les cases ont déjà quelque chose, l'insertion ne peut pas se faire et la fonction retourne un échec. Mais tant qu'il reste de la place, ça fonctionne bien, même s'il y a eu des collisions ou des suppressions avant.

1.5 Récupération d'un eléments

```c
void* hashmap_get(HashMap* map, const char* key) {
    unsigned long index = simple_hash(key);
    for (int i = 0; i < TABLE_SIZE; i++) {
        unsigned long pos = (index + i) % TABLE_SIZE;
        if (map->table[pos].key == NULL){
          return NULL;
        }
        if (strcmp(map->table[pos].key, key) == 0){
            return map->table[pos].value;
        }
    }
    return NULL;
}
```

Cette fonction récupère une valeur dans une table de hachage avec adressage ouvert. Elle cherche une clé dans la table, et si elle existe, elle retourne sa valeur. Sinon, elle retourne à NULL.

`unsigned long index = simple_hash(key);`

On calcule l'indice de départ dans la table grâce à une fonction de hachage.

Avec boucle for on va tenter au plus TABLE_SIZE essais (par probing linéaire), en cas de collision. On applique un adressage ouvert qui contient l'indice de départ + i (permet d'essayer les cases suivantes) % boucle autour si on dépasse la taille du tableau. Si on tombe sur une case vide, alors cela veut dire que la clé n'existe pas dans la table et on retourne NULL. Si la clé trouvée à cette position est égale à la clé recherchée, on retourne la valeur associée.

Si après avoir tout essayé, la clé n'est nulle part, on retourne NULL.

1.6 Suppression d'une clé d'une table de hachage

La fonction hashmap_remove permet de supprimer une clé (et sa valeur) d'une table de hachage. Elle commence par retrouver la position de la clé à l'aide de la fonction de hachage, puis elle cherche dans la table, en avançant case par case (probing linéaire), jusqu'à ce qu'elle tombe soit sur la clé recherchée, soit sur une case vide (auquel cas la clé n'existe pas dans la table).

Quand la clé est trouvée, elle est libérée de la mémoire, et la case est vidée (clé et valeur mises à NULL). Ensuite, la fonction fait quelque chose d'important : elle regarde les cases qui suivent, car elles peuvent contenir des éléments qui avaient été déplacés à cause d'une collision. Elle les réinsère proprement dans la table pour ne pas casser l'ordre de recherche. C'est une sorte de "réorganisation" locale.

Si la clé a été bien supprimée, la fonction retourne 1. Sinon, si elle n'a pas été trouvée, elle retourne 0.

1.7 Nettoyage de la mémoire utilisée

La fonction hashmap_destroy s'occupe de bien nettoyer toute la mémoire utilisée par la table de hachage. Elle commence par vérifier si la table existe. Si oui, elle parcourt chaque case de la table. Pour chaque case qui contient une clé, elle libère cette clé, et si la valeur associée correspond à l'un des registres utilisés (comme AX, BX, etc.), elle libère aussi cette valeur.

Il y a une petite vérification pour éviter de libérer quelque chose qu'il ne faut pas (par exemple un TOMBSTONE). Une fois que toutes les cases ont été traitées, elle libère la mémoire du tableau lui-même, puis celle de la structure principale de la table.

En résumé, c'est la fonction qui termine le travail proprement en libérant tout ce qui a été alloué.

1.8 Main d'exercice 1

Le fichier `main.c` contient le programme principal qui sert à tester le bon fonctionnement d'une structure de type HashMap, définie dans les fichiers d'implémentation. L'idée est de valider toutes les opérations fondamentales associées à une table de hachage : création, insertion, récupération, suppression et libération de mémoire.

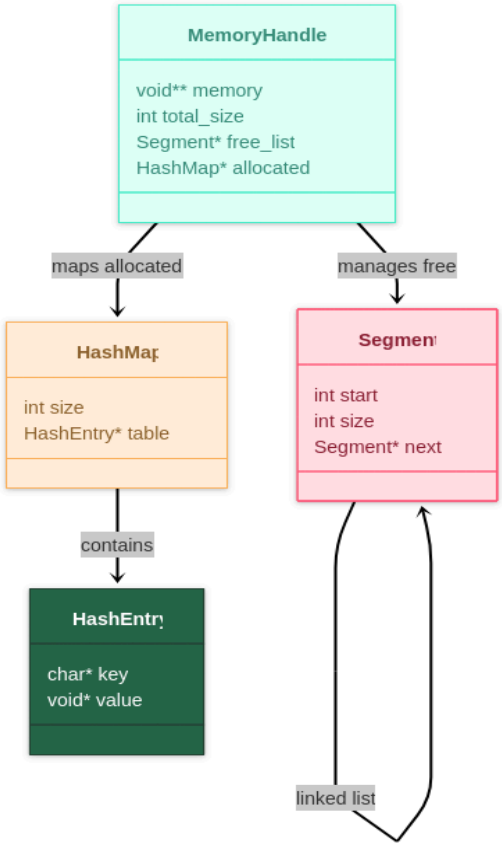
2-Gestion dynamique de la mémoire

Le gestionnaire de mémoire divise l'espace mémoire en segments de taille variable. Chaque segment peut être alloué à un programme ou libéré pour être réutilisé. L'allocation et la libération de segments suivent des étapes bien définies.

Dans notre cas, lorsqu'un segment est alloué, il est retiré de la liste des segments libres, puis il est ajouté à une table de hachage (correspondant aux segments alloués) en lui associant un nom unique. Lorsqu'un segment est libéré, il est supprimé de la table de hachage et réinséré dans la liste des segments libres (en le fusionnant avec d'autres segments libres adjacents s'il en existe, afin de limiter la fragmentation de la mémoire).

L'implémentation du gestionnaire reposera ainsi sur deux structures essentielles : la structure Segment représentant un bloc de mémoire (qui peut être libre ou alloué) et la structure MemoryHandler représentant l'état global du gestionnaire de mémoire.

2.1 Initialisation du gestionnaire de mémoire



La fonction `memory_init` initialise un gestionnaire de mémoire. Elle commence par allouer la structure principale, puis un tableau représentant la mémoire, une table de hachage pour suivre les allocations, et un premier segment libre qui occupe toute la mémoire. Si l'une des allocations échoue, tout est libéré proprement et `NULL` est retourné. Sinon, un pointeur vers le gestionnaire prêt à l'emploi est retourné.

2.2 Recherche d'un segment libre contenant une certaine plage mémoire

La fonction `find_free_segment` a pour objectif de localiser un segment libre dans la mémoire simulée, représentée par la liste chaînée `free_list` du gestionnaire de mémoire `handler`. Elle cherche un segment contenant entièrement la plage mémoire définie par l'adresse de début `start` et la taille `size`.

La recherche s'effectue de manière itérative : chaque segment de la liste est examiné, et si l'un d'eux commence avant ou au même point que `start`, et que sa fin dépasse ou atteint la fin de la plage souhaitée (`start + size`), alors ce segment est considéré comme suffisamment grand pour satisfaire la requête.

Si un segment adéquat est trouvé, un pointeur vers celui-ci est retourné. Par ailleurs, si l'argument `prev` est non nul, la fonction y place un pointeur vers le segment précédent, ce qui peut s'avérer utile pour des opérations ultérieures de mise à jour de la liste (comme la suppression ou le fractionnement du segment libre). Si aucun segment ne satisfait les conditions, la fonction retourne NULL.

2.3 Allocation d'un segment nommé à une position donnée

La fonction create_segment tente d'allouer un segment mémoire nommé, à une position et taille données, s'il existe un segment libre suffisant. Elle cherche un segment libre adéquat via find_free_segment . Si trouvé, elle crée un nouveau Segment avec malloc. Elle insère ce segment dans la table de hachage allocated.

Puis elle met à jour la liste free_list :

- soit en retirant le segment libre s'il correspond exactement,
- soit en le raccourcissant si partiellement utilisé.

Elle retourne 1 en cas de succès, 0 sinon.

2.4 Libération d'un segment nommé et fusion avec les segments libres

La fonction `remove_segment` libère un segment nommé elle récupère le segment via `hashmap_get`. Le retire de la table de hachage et aussi Le réinsère dans la liste `free_list` triée. À la fin, elle fusionne avec le segment précédent ou suivant si les adresses sont contiguës. Elle retourne 1 si le segment est libéré, 0 s'il n'existait pas.

2.5 Libération complète de la mémoire du gestionnaire

La fonction memory_destroy libère toute la mémoire gérée. On a décidé de fait cette fonction pour que l'on arrive de détruire la table de hachage des segments alloués ainsi libère le tableau de mémoire simulée. De plus libère chaque segment de la free_list. Enfin, libère la structure MemoryHandler elle-même.

3- Conception d’un parser pour un langage pseudo-assembleur

3.1 - parse_data_instruction 👍

```c
// Q3.1 - Fonction pour parser une instruction de la section .DATA
Instruction *parse_data_instruction(const char *line, HashMap *memory_locations) {
    Instruction *instruction = malloc(sizeof(Instruction));
    if (!instruction) return NULL;

    char *line_copy = strdup(line); // Dupliquer la ligne pour la manipulation
    char *mnemonic = strtok(line_copy, " ");
    char *operand1 = strtok(NULL, " ");
    char *operand2 = strtok(NULL, " ");

    instruction->mnemonic = strdup(mnemonic);
    instruction->operand1 = operand1 ? strdup(operand1) : NULL;
    instruction->operand2 = operand2 ? strdup(operand2) : NULL;

    // Associer la variable à son adresse
    int address = memory_locations->size; // Exemple d'adresse
    hashmap_insert(memory_locations, mnemonic, (void *)(long)address);

    free(line_copy); // Libérer la copie de la ligne
    return instruction;
}
```

Cette fonction prend une ligne de .DATA comme "X DW 10" et la transforme en une structure Instruction.

- On duplique la ligne pour pas toucher à l’originale,
- On découpe en 3 morceaux : nom, type, valeur,
- On découpe en 3 morceaux : nom, type, valeur,
- On stocke le nom et les deux opérandes dans une structure qu’on alloue à la main,
- Puis on associe la variable à une adresse unique dans memory_locations, basée sur le nombre d'éléments déjà stockés.

3.2 - parse_code_instruction 👍

```c
// Q3.2 - instruction permettant d'analyser et stocker une ligne de la section.code
Instruction *parse_code_instruction(const char *line, HashMap *labels, int code_count) {
    Instruction *instruction = malloc(sizeof(Instruction));
    if (!instruction) return NULL;

    char *line_copy = strdup(line); // Dupliquer la ligne pour la manipulation
    char *label = strtok(line_copy, ":");
    if (label) {
        // Si une étiquette est présente
        hashmap_insert(labels, label, (void *)(long)code_count);
        strtok(NULL, ":"); // Passer à la prochaine partie après le label
    }

    char *mnemonic = strtok(NULL, " ");
    if (!mnemonic) {
        free(line_copy);
        free(instruction);
        return NULL; // Si le mnémotechnique est absent, retourner NULL
    }

    char *operand1 = strtok(NULL, ",");
    char *operand2 = strtok(NULL, ",");

    instruction->mnemonic = strdup(mnemonic);
    instruction->operand1 = operand1 ? strdup(operand1) : NULL; // Ne pas allouer si NULL
    instruction->operand2 = operand2 ? strdup(operand2) : NULL; // Ne pas allouer si NULL

    free(line_copy); // Libérer la copie de la ligne
    return instruction;
}
```

Cette fonction prend une ligne du .CODE, du type : MOV AX,7 ou loop: JMP end

- Si la ligne commence par un **label**, on l’enregistre dans labels avec code_count comme index,
- Ensuite on récupère les **mnémotechniques et opérandes** séparés par des espaces et virgules,
- On duplique les chaînes et on stocke tout dans une structure Instruction.

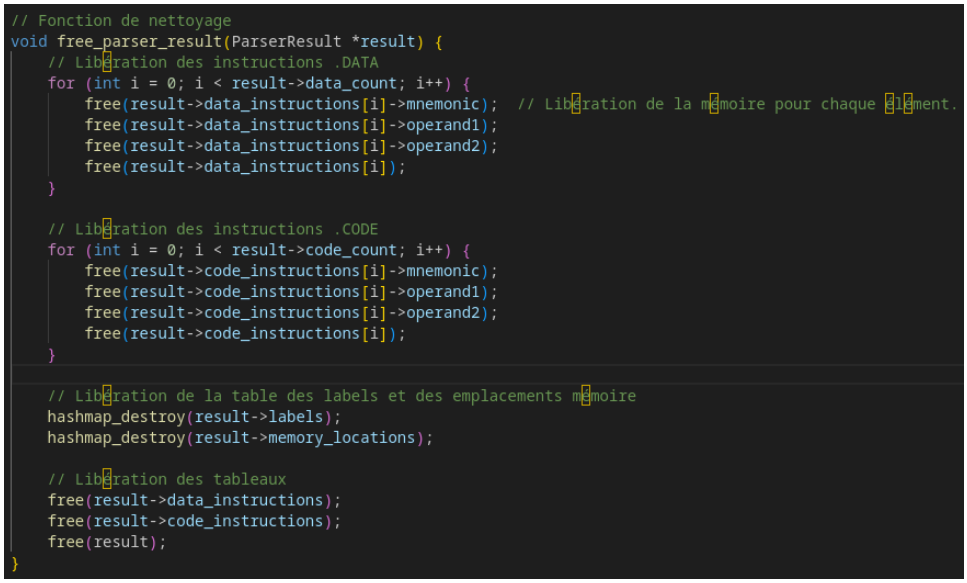
3.3 - parse 👍

La fonction parse lit tout le fichier assembleur et sépare automatiquement les sections .DATA et .CODE.

- Elle initialise la structure ParserResult avec des tableaux vides,
- Puis elle **lit ligne par ligne**, et appelle soit parse_data_instruction, soit parse_code_instruction,
- Les instructions sont stockées dynamiquement avec realloc.

C'est le cœur du parser : il transforme un fichier texte en une vraie structure prête à être exploitée.

3.4 - free_parser_result 👍



```c
// Fonction de nettoyage
void free_parser_result(ParserResult *result) {
    // Libération des instructions .DATA
    for (int i = 0; i < result->data_count; i++) {
        free(result->data_instructions[i]->mnemonic);  // Libération de la mémoire pour chaque élément.
        free(result->data_instructions[i]->operand1);
        free(result->data_instructions[i]->operand2);
        free(result->data_instructions[i]);
    }

    // Libération des instructions .CODE
    for (int i = 0; i < result->code_count; i++) {
        free(result->code_instructions[i]->mnemonic);
        free(result->code_instructions[i]->operand1);
        free(result->code_instructions[i]->operand2);
        free(result->code_instructions[i]);
    }

    // Libération de la table des labels et des emplacements mémoire
    hashmap_destroy(result->labels);
    hashmap_destroy(result->memory_locations);

    // Libération des tableaux
    free(result->data_instructions);
    free(result->code_instructions);
    free(result);
}
```

Nettoyage complet de tout ce qui a été alloué :

- Chaque `Instruction` est libérée champ par champ,
- Les tables de hachage sont détruites avec `hashmap_destroy`,
- Et on termine par les tableaux et la structure principale.

Important pour éviter les fuites mémoire, surtout dans un projet avec parsing dynamique.

4-Allocation d’un segment de données

4.1 Fonction permettant d'initialiser le processeur

La fonction cpu_init initialise une structure CPU. Elle alloue la mémoire pour la structure CPU. Apres initialisation de la mémoire du CPU avec memory_init, puis réer une table de hachage pour les registres (contexte). Insère les registres "AX", "BX", "CX", "DX" avec la même adresse de la variable initial_value.

4.2 Libération de toutes les ressources allouées

Cette fonction libère toute la mémoire allouée pour un processeur simulé.

Elle fait trois choses :

1. **Libère les registres** : hashmap_destroy(cpu->context) détruit la table de hachage contenant les registres comme "AX", "BX", etc.
2. **Libère la mémoire** : memory_destroy(cpu->memory_handler) libère la mémoire simulée et les segments.
3. **Libère la structure CPU elle-même** : free(cpu).

4.3 Stocker data

```
void *store(MemoryHandler *handler, const char *segment_name, int pos, void *data) {
    Segment *segment = hashmap_get(handler->allocated, segment_name);
    if (!segment || pos < 0 || pos >= segment->size) {
        return NULL; // Segment non trouvé ou position invalide
    }
    handler->memory[segment->start + pos] = data; // Stocker la donnée à la position
spécifiée
    return data;
}
```

Cette fonction stocke une donnée dans un segment mémoire précis.

Voici ce qu’elle fait :

1. **Elle récupère le segment alloué** dont le nom est segment_name (dans la table de hachage).
2. **Elle vérifie** que le segment existe et que pos est une position valide (entre 0 et segment->size - 1).
3. **Elle stocke la donnée** dans handler->memory à l’adresse calculée : segment->start + pos.

4.4 Récupération des données stockées

```
void *load(MemoryHandler *handler, const char *segment_name, int pos) {
    Segment *segment = hashmap_get(handler->allocated, segment_name);
    if (!segment || pos < 0 || pos >= segment->size) {
        return NULL; // Segment non trouvé ou position invalide
    }
    return handler->memory[segment->start + pos]; // Récupérer la donnée à la position
spécifiée
}
```

Cette fonction permet **de lire une donnée** depuis un segment mémoire identifié par son nom.

1. Elle cherche le segment dans la table de hachage via son nom (segment_name).
2. Elle vérifie si le segment existe et si la position demandée (pos) est valide.
3. Si tout est correct, elle retourne la donnée à l'adresse segment->start + pos dans le tableau handler->memory.

4.5 Allocation dynamiquement le segment de données

La fonction allocate_variables initialise un segment mémoire nommé "DS" dans le but de stocker des variables issues d'un tableau d'instructions. Elle calcule d'abord la taille totale requise (chaque variable occupant une unité), puis crée le segment via create_segment. Ensuite, elle alloue dynamiquement la mémoire pour chaque valeur (extraites des opérandes de l'instruction), les convertit en entiers, et les stocke dans le segment à l'aide de la fonction store.

4.6 L'afficher le contenu

La fonction `print_data_segment` permet de visualiser le contenu du segment `"DS"`.

```
void print_data_segment(CPU *cpu) {
    Segment *segment = hashmap_get(cpu->memory_handler->allocated, "DS");
    if (!segment) {
        printf("Segment de données non trouvé.\n");
        return;
    }

    printf("Contenu du segment de données (DS):\n");
    for (int i = 0; i < segment->size; i++) {
        int *data = (int *)load(cpu->memory_handler, "DS", i);
        if (data) {
            printf("Position %d: %d\n", i, *data);
        }
    }
}
```

Elle accède au segment via la table de hachage des segments alloués, puis itère sur chaque position du segment. À chaque itération, elle utilise `load` pour récupérer la donnée correspondante, qu'elle affiche ensuite si elle est non nulle.

5-Expressions régulières et résolution d’adressage

5.1 Initialisation de la CPU2

La fonction cpu_init_2 initialise une structure CPU2 représentant un processeur simulé avec mémoire, registres et une "constant pool".

- **Allocation et initialisation :**
  Elle commence par allouer dynamiquement un espace mémoire pour la structure CPU2. En cas d’échec, elle retourne à NULL.
- **Sous-structures :**
  - Le gestionnaire de mémoire (memory_handler) est initialisé via memory_init.
  - Deux tables de hachage sont créées : l’une pour les **registres** (context), l’autre pour la **constant pool**.
- **Registres simulés :**
  Sept registres sont alloués et initialisés à zéro : AX, BX, CX, DX, IP (Instruction Pointer), ZF (Zero Flag), et SF (Sign Flag).
  Chaque registre est ensuite inséré dans la table de hachage context.
- **Retour :**
  La fonction retourne un pointeur vers la structure CPU2 entièrement configurée.

5.2 Destruction de la CPU2

free_segment_data est responsable de la libération des données d'un segment de mémoire. Elle prend en entrée un MemoryHandler et le nom d'un segment. Voici les étapes qu'elle suit :

- **Récupération du segment** : Elle utilise hashmap_get pour obtenir le segment de mémoire correspondant au nom fourni. Si le segment existe, elle continue.
- **Libération des données** : Pour chaque élément du segment, elle charge les données via la fonction load (qui est présumée récupérer les données à un index spécifique du segment). Si les données sont présentes, elles sont libérées avec free(data).
- **Affichage** : Elle affiche un message pour chaque index libéré, indiquant l'index et le nom du segment.

2. Fonction cpu_destroy

Cette fonction détruit une instance de la structure CPU2, en effectuant plusieurs libérations de mémoire pour les différentes parties du processeur. Les étapes de la fonction sont les suivantes :

- **Libération des segments** : Elle commence par libérer les données du segment DS (Data Segment) en appelant free_segment_data.
- **Destruction des hashmap** : Elle libère la mémoire associée aux registres (stockés dans des hashmap) via hashmap_destroy. Cela supprime également toutes les données associées aux clés dans ces hashmap.
- **Libération du gestionnaire de mémoire** : La mémoire associée au gestionnaire de mémoire (via cpu->memory_handler) est détruite par la fonction memory_destroy.
- **Libération de la CPU** : Enfin, l'objet cpu lui-même est libéré avec free(cpu).

5.3 Adressage immédiat

La fonction `immediate_addressing` permet de gérer de manière efficace les opérandes constantes dans le simulateur de processeur, en utilisant un `constant_pool` pour optimiser la gestion de la mémoire et éviter les répétitions. Le traitement des opérandes sous forme d'entiers est bien conçu, avec une vérification du format et une insertion dans un cache. Cependant, des améliorations peuvent être apportées en matière de gestion des erreurs et de la mémoire.

```
void *immediate_addressing(CPU2 *cpu, const char *operand) {
    const char *pattern = "^[0-9]+$";
    if (matches(pattern, operand)) {
        void *value = hashmap_get(cpu->constant_pool, operand);
        if (!value) {
            int *num = (int *)malloc(sizeof(int));
            *num = atoi(operand);
            hashmap_insert(cpu->constant_pool, operand, num);
            return num;
        }
        return value;
    }
    return NULL;
}
```

5.4 Adressage par registre

La fonction register_addressing permet de récupérer la valeur d'un registre dans un processeur simulé. Elle vérifie si l'opérande est l'un des registres valides (AX, BX, CX, DX) via une expression régulière. Si c'est le cas, elle retourne la valeur du registre à partir de cpu->context. Si l'opérande n'est pas un registre valide, elle retourne NULL.

5.5 Adressage direct en mémoire

La fonction memory_direct_addressing permet d'effectuer un adressage direct en mémoire. Elle vérifie si l'opérande correspond à un format d'adresse mémoire, comme [123], via une expression régulière. Si l'opérande est valide, elle extrait l'adresse, puis charge la valeur de cette adresse à partir du segment "DS" en utilisant la fonction load. Si l'opérande n'est pas une adresse valide, la fonction retourne NULL.

5.6 Adressage indirect par registre

La fonction register_indirect_addressing effectue un adressage indirect en utilisant un registre (comme [AX], [BX], etc.). Elle vérifie si l'opérande est dans le format attendu, extrait le nom du registre sans les crochets, puis récupère la valeur du registre via register_addressing. Si la valeur du registre est valide, elle est utilisée comme une adresse pour charger la valeur correspondante en mémoire dans le segment "DS". Si l'opérande n'est pas valide ou s'il y a un problème lors du chargement de la valeur, un message d'erreur est affiché.

5.7 Gestion de l'instruction MOV

La fonction handle_MOV effectue une opération de copie de valeur entre deux emplacements mémoire, src et dest. Voici son fonctionnement :

1. **Paramètre inutilisé** : Le paramètre cpu est marqué comme non utilisé avec (void)cpu;, ce qui peut être utile dans certains contextes de compilation pour éviter les avertissements si le paramètre n'est pas utilisé dans la fonction.
2. **Vérification des pointeurs** : La fonction vérifie si les deux pointeurs src et dest ne sont pas NULL. Si l'un d'eux est NULL, l'opération ne se fait pas.
3. **Copie de la valeur** : Si les pointeurs sont valides, la fonction copie la valeur entière pointée par src dans l'emplacement pointé par dest. Cela est effectué en supposant que src et dest pointent vers des variables de type int.

5.8 Résolution d'adressage

La fonction resolve_addressing essaie de résoudre un opérande en utilisant différents modes d'adressage. Elle commence par l'adressage immédiat, puis teste l'adressage par registre, l'adressage direct en mémoire et enfin l'adressage indirect par registre. Si un mode d'adressage est applicable, elle retourne la valeur correspondante. Si aucun mode n'est applicable, elle retourne à NULL.

6- Allocation d’un segment de codes et exécution

6.1 trim

Cette fonction nettoie une chaîne en supprimant les **caractères invisibles** (espaces, tab, saut de ligne, retour chariot) **au début et à la fin**.

1. On avance le pointeur `str` tant qu'on tombe sur des espaces au début.
2. Ensuite, on cherche l'extrémité de la chaîne (`end`), et on coupe tout ce qui suit les espaces à la fin.
3. On retourne la nouvelle chaîne nettoyée.

Utile pour préparer proprement les lignes avant parsing.

6.2 search_and_replace

Objectif : Remplacer des noms de variables ou labels dans une chaîne `*str` par leur valeur numérique venant d'une table de hachage.

1. On vérifie les entrées. Si `str`, `*str` ou `values` sont NULL, on retourne direct.
2. On parcourt chaque case de la table `values`.
3. Pour chaque **clé (key)** présente, on cherche si elle est **contenue dans la chaîne `*str`** avec `strstr`.
4. Si c'est le cas :
    - On crée une chaîne temporaire `replacement` contenant la **valeur de la clé**, en version texte (`snprintf`).
    - On reconstruit une **nouvelle chaîne** avec la partie avant la clé, puis la valeur, puis la fin.
    - On remplace l'ancienne chaîne, on la libère et on met à jour le pointeur.
5. On retourne 1 si au moins un remplacement a été fait.

6.2 resolve_constants

Cette fonction applique `search_and_replace` sur toutes les instructions du bloc `.CODE`.
Elle fait le travail de traduction symbolique → adresse mémoire.

1. Pour chaque instruction du code :
    - Si `operand1` existe, on essaie de la remplacer via `memory_locations`, puis `labels`.
    - Pareil pour `operand2`.
2. Si un remplacement est effectué, on note que le code a été modifié (`modified = 1`).
3. On retourne 1 si quelque chose a changé

Essentiel pour convertir un code lisible humainement en code prêt pour la simulation CPU.

6.3 cpu_inite

Crée et initialise un **simulateur CPU** (CPU2).

1. Allocation dynamique de la structure CPU2.
2. Création du gestionnaire mémoire (`memory_init`) avec la taille donnée.
3. Création des hashmaps :
   - `context` : pour stocker les registres (AX, BX…),
   - `constant_pool` : pour les constantes à venir.
4. Initialisation des registres de base (AX à SF) à 0.

Le CPU est maintenant prêt à exécuter un programme.

6.4 allocate_code_segment

Cette fonction **crée le segment mémoire "CS"**, et y range les instructions du programme.

1. On appelle `create_segment` avec `"CS"` pour allouer de la place.

   Chaque instruction est stockée à l'adresse correspondante (index `i`) avec `store`.

   On réinitialise le registre `IP` à 0 (départ du programme).

On a une mémoire prête à lancer l'exécution.

6.5 handle_instruction

Fonction qui **interprète et exécute une instruction**, en modifiant l'état du CPU selon le mnémonique.

- `MOV` copie la valeur `src` vers `dst`.
- `JMP` change `IP` vers `src` (saut direct).
- `JZ` saute si le flag `ZF` vaut 1.
- `JNZ` saute si `ZF` vaut 0.
- `HALT` termine le programme en déplaçant `IP` à la fin du code.

C'est ici que les instructions prennent vie

6.6 execute_instruction

On résout les opérandes (`operand1` → destination, `operand2` → source), puis on les envoie à `handle_instruction`.

C'est le **pont entre le texte de l'instruction et sa logique réelle**.

6.7 fetch_next_instruction

Cherche l'instruction située à `IP` dans le segment `"CS"`, l'incrémente et retourne l'instruction trouvée.

Permet de parcourir le programme **pas à pas**.

6.8 run_program

Boucle d'exécution complète :

1. Affiche l'état initial du CPU.
2. Exécute chaque instruction une à une.
3. Affiche l'instruction, attend entrée de l'utilisateur.
4. S'arrête sur `HALT`, erreur ou demande utilisateur.

Super utile pour faire **du pas-à-pas**, parfait en contexte pédagogique.

7-Implémentation d’un segment de pile (Stack Segment)

Dans cet exercice, nous nous intéressons au segment de pile, nomm´e ”SS” (pour ”stack segment”), qui est une zone m´emoire d´edi´ee `a la gestion dynamique des donn´ees temporaires durant l’ex´ecution d’un programme. Sa manipulation repose principalement sur deux registres essentiels :

— SP (pour ”Stack Pointer”) : pointe toujours vers le sommet de la pile, indiquant la dernière adresse utiliser. Chaque op´eration ”PUSH” décrémente SP, tandis que chaque op´eration ”POP” l'incrément.

— BP (pour ”Base Pointer”) : fournit un point de référence stable pour accéder aux paramètres et variables locales d’une fonction, servant de base pour le référencement des données sur la pile.

Dans notre cas, il s’agit d’un pointeur sur le bas de la pile, c’est-`a-dire l’adresse mémoire la plus grande (car la pile s'étend vers les adresses mémoire inférieures).

7.1 Initialisation de la pile (Stack)

Cette fonction initialise la structure CPU, crée la mémoire et configure les registres nécessaires à la gestion de la pile. Elle crée un segment de pile, initialise les registres SP et BP, et retourne un pointeur vers la structure CPU initialisée.

7.2 Empiler une valeur

La fonction empile une valeur entière dans la pile. Elle vérifie d'abord que la pile n'est pas pleine (dépassement de pile), puis stocke la valeur à l'adresse pointée par SP et décrémente SP.

La fonction `pop_value` dépile une valeur depuis la pile. Elle vérifie que la pile n'est pas vide (sous-flux de pile), incrémente SP, récupère la valeur de la pile, et la place dans une variable de destination.

7.3 Dépiler une valeur

Cette fonction gère les instructions PUSH et POP. Pour PUSH, elle empile la valeur de l'opérande (soit un registre, soit une valeur immédiate). Pour POP, elle dépile la valeur et la place dans un registre spécifié. Elle gère également les erreurs de débordement et sous-flux de pile.

8- Gestion de l’allocation dynamique (Extra Segment)

8.1 segment_override_addressing

Cette fonction gère une forme spéciale d’adressage du type `[ES:AX]`, c’est-à-dire :
→ "va chercher dans le segment ES à l'adresse contenue dans le registre AX".

1. Elle vérifie que le CPU et l’opérande ne sont pas `NULL`.
2. Elle utilise une expression régulière pour détecter la syntaxe `[SEGMENT:REGISTRE]`.
3. Si le format est reconnu :
    - Elle extrait le **nom du segment** (ex : ES) et du **registre** (ex : AX),
    - Elle récupère la **valeur du registre** dans `cpu->context`,
    - Puis elle accède à la mémoire via `load(...)`, à partir du segment et de l’offset.

Utile pour accéder dynamiquement à des cases mémoire avec précision.

8.2 resolve_addressing_2

C’est une **extension** de `resolve_addressing` pour supporter le format `[ES:AX]`.

1. Elle regarde si l’opérande contient un `:`.
   → Si oui, elle suppose qu’il s’agit d’un adressage spécial,
2. Elle appelle `segment_override_addressing`.
3. Sinon, on laisse la résolution classique.

Gère automatiquement tous les types d’adressage.

8.3 find_free_address_strategy

Cherche un **espace libre en mémoire** selon une stratégie d’allocation :

- First Fit (premier bloc suffisant),
- Best Fit (bloc qui gaspille le moins),
- Worst Fit (bloc le plus large possible).

1. Elle vérifie les paramètres : taille positive et pointeur mémoire valide.
2. Elle parcourt la liste des blocs libres.
3. Selon la stratégie choisie (valeur dans BX souvent), elle sélectionne un bloc.

Retourne l'adresse de début du bloc, ou `-1` si rien trouvé.

8.4 ajout_ES_dans_cpu_init

Ajoute le **registre ES** dans le CPU.
On l'initialise à `-1` pour dire que **rien n'a encore été alloué**.

→ On traite ES comme un registre à part entière, exactement comme AX ou BX.

Nécessaire pour l'allocation dynamique avec `ALLOC`.

8.5 alloc_es_segment

Cette fonction alloue dynamiquement un segment `"ES"`.

1. Elle lit :
   - **AX** → la taille souhaitée du segment,
   - **BX** → la stratégie à utiliser,
   - **ZF** et **ES** pour mise à jour du CPU.
2. Elle cherche une zone libre avec `find_free_address_strategy`.
3. Si aucune zone trouvée → `ZF = 1`, retourne 0 (échec).
4. Sinon :
   - Crée le segment avec `create_segment`,
   - Initialise les cases à 0,
   - Stocke l'adresse dans le registre `ES`,
   - Met `ZF = 0` et retourne 1.

Permet au CPU de **réserver dynamiquement un bloc mémoire** en cours d'exécution.

8.6 free_es_segment

Libère le segment `"ES"` précédemment alloué.

1. Récupère l'adresse depuis le registre `ES`.
2. Si ES vaut `-1` → aucun segment à libérer → on retourne -1.
3. Sinon :
    - Récupère le segment depuis la mémoire,
    - Libère chaque valeur,
    - Supprime le segment avec `remove_segment`,
    - Réinitialise ES à `-1`.

Permet une gestion propre de la mémoire dynamique.

8.7 handle_instruction_extended

Étend `handle_instruction` pour gérer les nouvelles instructions :

- `"ALLOC"` → appelle `alloc_es_segment`,
- `"FREE"` → appelle `free_es_segment`.

Si ce n'est pas l'un de ces deux :

1. Elle crée une instruction temporaire,
2. Appelle `handle_instruction` normalement,
3. Libère les champs à la fin.

Rend possible l'utilisation d'instructions supplémentaires dans l'assembleur.